Module 3: Packages, Exception Handling and Multithreading

Premanand S

Assistant Professor
School of Electronics Engineering
Vellore Institute of Technology
Chennai Campus

premanand.s@vit.ac.in

March 23, 2026

Motivation: Why Multithreading?

Real Life Example: Smartphone

- Listening to music

Motivation: Why Multithreading?

Real Life Example: Smartphone

- Listening to music
- Downloading a file

Motivation: Why Multithreading?

Real Life Example: Smartphone

- Listening to music
- Downloading a file
- Chatting on WhatsApp

Motivation: Why Multithreading?

Real Life Example: Smartphone

- Listening to music
- Downloading a file
- Chatting on WhatsApp
- Receiving notifications

Motivation: Why Multithreading?

Real Life Example: Smartphone

- Listening to music
- Downloading a file
- Chatting on WhatsApp
- Receiving notifications

Observation

Multiple tasks run simultaneously.

Motivation: Why Multithreading?

Real Life Example: Smartphone

- Listening to music
- Downloading a file
- Chatting on WhatsApp
- Receiving notifications

Observation

Multiple tasks run simultaneously.

In programming this concept is called **Multithreading**.

Why Do We Need Synchronization?

A bank system once crashed...

Why Do We Need Synchronization?

A bank system once crashed...
because two employees updated the same account

Why Do We Need Synchronization?

A bank system once crashed...
because two employees updated the same account
at the same time...

Why Do We Need Synchronization?

A bank system once crashed...
because two employees updated the same account
at the same time...

Money got duplicated.

Why Do We Need Synchronization?

A bank system once crashed...
because two employees updated the same account
at the same time...

Money got duplicated.

How did this happen?

Why Do We Need Synchronization?

A bank system once crashed...
because two employees updated the same account
at the same time...

Money got duplicated.

How did this happen?

Today we'll learn why that happens...

Why Do We Need Synchronization?

A bank system once crashed...
because two employees updated the same account
at the same time...

Money got duplicated.

How did this happen?

Today we'll learn why that happens...
and how to prevent it using

Why Do We Need Synchronization?

A bank system once crashed...
because two employees updated the same account
at the same time...

Money got duplicated.

How did this happen?

Today we'll learn why that happens...
and how to prevent it using
Synchronization and Inter-Thread Communication

Introduction to Multithreading

Right now... you are listening to me,

Introduction to Multithreading

Right now... you are listening to me,
breathing,

Introduction to Multithreading

Right now... you are listening to me,
breathing,
thinking,

Introduction to Multithreading

Right now... you are listening to me,
breathing,
thinking,
and maybe even planning lunch...

Introduction to Multithreading

Right now... you are listening to me,
breathing,
thinking,
and maybe even planning lunch...

Your brain is doing multiple tasks simultaneously...

Introduction to Multithreading

Right now... you are listening to me,
breathing,
thinking,
and maybe even planning lunch...

Your brain is doing multiple tasks simultaneously...

So tell me... can a computer do the same?

Introduction to Multithreading

Right now... you are listening to me,
breathing,
thinking,
and maybe even planning lunch...

Your brain is doing multiple tasks simultaneously...

So tell me... can a computer do the same?

That ability in computers is called Multithreading.

What is a Thread?

- A **Thread** is the smallest unit of execution inside a program.

What is a Thread?

- A **Thread** is the smallest unit of execution inside a program.
- A program may contain multiple threads.

What is a Thread?

- A **Thread** is the smallest unit of execution inside a program.
- A program may contain multiple threads.
- Each thread performs a separate task.

What is a Thread?

- A **Thread** is the smallest unit of execution inside a program.
- A program may contain multiple threads.
- Each thread performs a separate task.
- Threads run concurrently within the same process.

Process vs Thread

Process	**Thread**
Independent program	Part of a process
Heavyweight	Lightweight
Separate memory space	Shared memory space
Example: Browser	Example: Tabs in browser

Single Thread vs Multithread

Single Thread

- Tasks execute one after another
- Slower execution

Single Thread vs Multithread

Single Thread

- Tasks execute one after another
- Slower execution

Multithreading

- Multiple tasks execute simultaneously
- Better CPU utilization
- Faster program execution

Single Thread Execution Example

Java Code

```
class Main {
 public static void task() {
  for (int i = 0; i < 5; i++) {
   System.out.println(i);
   try {
    Thread.sleep(1000);
   } catch (InterruptedException e) {
    e.printStackTrace();
   }
  }
 }

 public static void main(String[] args) {
  task();
  task();
 }
}
```

Multithreading Example (Concurrent Execution)

Java Code

```
class MyThread extends Thread {
 public void run() {
  for (int i = 0; i < 5; i++) {
   System.out.println(i);
   try {
    Thread.sleep(1000);
   } catch (InterruptedException e) {
    e.printStackTrace();
   } } } }
class Main {
 public static void main(String[] args) {
  MyThread t1 = new MyThread();
  MyThread t2 = new MyThread();

  t1.start();
  t2.start();
 } }
```

Thread Life Cycle

1. New

Thread Life Cycle

1. New
2. Runnable

Thread Life Cycle

1. New
2. Runnable
3. Running

Thread Life Cycle

1. New
2. Runnable
3. Running
4. Waiting / Blocked

Thread Life Cycle

1. New
2. Runnable
3. Running
4. Waiting / Blocked
5. Terminated

New State

Definition

- Thread object is created
- Thread has not started execution

Code Example

```
class MyThread extends Thread {
 public void run(){
  System.out.println("Thread running");
 }
}
public class Test {
 public static void main(String[] args){
  MyThread t1 = new MyThread();

  // Thread created but not started
 }
}
```

Runnable State

Definition

- Thread becomes ready for execution
- Waiting for CPU scheduling

Code Example

```
public class Test {

 public static void main(String[] args){

  MyThread t1 = new MyThread();

  t1.start(); // Thread enters Runnable state

 }

}
```

Running State

Definition

- CPU allocates time to the thread
- run() method starts execution

Code Example

```
class MyThread extends Thread {

 public void run(){

  for(int i=1;i<=5;i++)
  System.out.println("Thread running: " + i);

 }

}
```

Waiting / Blocked State

Definition

- Thread temporarily stops execution
- Waiting for resource or time delay

Example Methods

- sleep()
- wait()
- join()

Code Example

```
class MyThread extends Thread {
 public void run(){
  try{
   Thread.sleep(2000);
  }
  catch(Exception e){}
  System.out.println("Thread resumed");
 }
}
```

Terminated State

Definition

- Thread completes execution
- run() method finishes
- Thread cannot be restarted

Code Example

```
class MyThread extends Thread {

 public void run(){

  System.out.println("Thread finished");

 }

}
```

Creating Threads in Java

Java provides two ways to create threads.

Creating Threads in Java

Java provides two ways to create threads.

Method 1

- Extending **Thread** class

Creating Threads in Java

Java provides two ways to create threads.

Method 1

- Extending **Thread** class

Method 2

- Implementing **Runnable** interface

Thread Creation using Thread Class

```
class MyThread extends Thread {
 public void run(){
  System.out.println("Thread running");
 }
}
public class Test {
 public static void main(String[] args){
  MyThread t1 = new MyThread();
  t1.start();
 }
}
```

start() vs run()

start()	**run()**
Creates new thread	Normal method call
Executes concurrently	Executes sequentially
Calls run() internally	Runs in main thread

Thread Creation using Runnable Interface

```
class MyThread implements Runnable {
 public void run(){
  System.out.println("Thread running");
 }
}
public class Test {
 public static void main(String[] args){
  MyThread obj = new  MyThread();
  Thread t1 = new Thread(obj);
  t1.start();
 }
}
```

Thread Class vs Runnable Interface

Thread Class	**Runnable Interface**
Uses inheritance	Uses interface
Cannot extend another class	Can extend another class
Less flexible	More flexible

Creating Multiple Threads

```
class MyThread extends Thread {
 public void run(){
  for(int i=1;i<=5;i++)
  System.out.println(i);
 }
}
public class Test {
 public static void main(String[] args){
  MyThread t1 = new MyThread();
  MyThread t2 = new MyThread();

  t1.start();
  t2.start();
 }
}
```

Problem: Shared Resource

When multiple threads access the same resource:

Problem: Shared Resource

When multiple threads access the same resource:

- Data inconsistency may occur

Problem: Shared Resource

When multiple threads access the same resource:

- Data inconsistency may occur
- This situation is called a **Race Condition**

Race Condition Example

Shared Resource

```
class Counter {
 int value = 0;
}
```

Race Condition Example

Shared Resource

```
class Counter {
 int value = 0;
}
```

Thread Execution

```
class MyThread extends Thread {
 Counter counter;
 MyThread(Counter counter) {
  this.counter=counter;}
 public void run() {
  for (int i = 0; i < 100000; i++) {
   counter.value++; // NOT safe
  }  }   }
```

Execution of Threads

Main Method

```
public class Main {
 public static void main(String[] args)
 throws InterruptedException {
  Counter counter = new Counter();
  MyThread t1 = new MyThread(counter);
  MyThread t2 = new MyThread(counter);
  t1.start();
  t2.start();
  t1.join();
  t2.join();
  System.out.println(counter.value);
 }  }
```

Expected Output:

Race Condition

Expected Output:

- 200000

Race Condition

Expected Output:

- 200000

Actual Output:

Race Condition

Expected Output:

- 200000

Actual Output:

Reason:

Race Condition

Expected Output:

- 200000

Actual Output:

Reason:

- Multiple threads access shared data

Race Condition

Expected Output:

- 200000

Actual Output:

Reason:

- Multiple threads access shared data
- Operation is not atomic

Race Condition

Expected Output:

- 200000

Actual Output:

Reason:

- Multiple threads access shared data
- Operation is not atomic

This problem is called Race Condition

Synchronization

Synchronization

- Ensures only one thread accesses critical section at a time

Synchronization

Synchronization

- Ensures only one thread accesses critical section at a time
- Prevents data inconsistency

Synchronization

Synchronization

- Ensures only one thread accesses critical section at a time
- Prevents data inconsistency
- Achieved using **synchronized** keyword

Fixing Race Condition using Synchronization

Solution: Use synchronized method

Fixing Race Condition using Synchronization

Solution: Use synchronized method

Modified Counter Class

```
class Counter {
 int value = 0;

 synchronized void increment() {
  value++;
 }
}
```

Fixing Race Condition using Synchronization

Solution: Use synchronized method

Modified Counter Class

```
class Counter {
 int value = 0;

 synchronized void increment() {
  value++;
 }
}
```

Only one thread can access increment() at a time

Question:

Inter-Thread Communication

Question:
How do threads communicate with each other?

Inter-Thread Communication

Question:
How do threads communicate with each other?

Threads sometimes need to coordinate their execution...

Inter-Thread Communication

Question:
How do threads communicate with each other?

Threads sometimes need to coordinate their execution...

Java provides special methods:

Inter-Thread Communication

Question:
How do threads communicate with each other?

Threads sometimes need to coordinate their execution...

Java provides special methods:

- `wait()`

Inter-Thread Communication

Question:
How do threads communicate with each other?

Threads sometimes need to coordinate their execution...

Java provides special methods:

- `wait()`
- `notify()`

Inter-Thread Communication

Question:
How do threads communicate with each other?

Threads sometimes need to coordinate their execution...

Java provides special methods:

- `wait()`
- `notify()`
- `notifyAll()`

Inter-Thread Communication

Question:
How do threads communicate with each other?

Threads sometimes need to coordinate their execution...

Java provides special methods:

- `wait()`
- `notify()`
- `notifyAll()`

wait() Method in Java

What is wait()?

wait() Method in Java

What is wait()?

- Pauses the execution of a thread

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

When is it used?

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

When is it used?

- When a thread must wait for a condition

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

When is it used?

- When a thread must wait for a condition
- Example: waiting for data to be available

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

When is it used?

- When a thread must wait for a condition
- Example: waiting for data to be available

Important Rules:

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

When is it used?

- When a thread must wait for a condition
- Example: waiting for data to be available

Important Rules:

- Must be used inside **synchronized block**

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

When is it used?

- When a thread must wait for a condition
- Example: waiting for data to be available

Important Rules:

- Must be used inside **synchronized block**
- Requires another thread to call `notify()`

wait() Method in Java

What is wait()?

- Pauses the execution of a thread
- Thread enters **waiting state**
- Releases the lock on the object

When is it used?

- When a thread must wait for a condition
- Example: waiting for data to be available

Important Rules:

- Must be used inside **synchronized block**
- Requires another thread to call `notify()`

wait() and notify() Example

Java Code

```
public class Main {
 public static void main(String[] args)
 throws InterruptedException {
  final Object lock = new Object();
  Thread t1 = new Thread(() -> {
   synchronized (lock) {
    System.out.println("Thread 1: Waiting...");
    try {
     lock.wait();
    } catch (InterruptedException e) {}
    System.out.println("Thread 1: Resumed!");
   } });
```

wait() and notify() Example

Java Code

```
    Thread t2 = new Thread(() -> {
     synchronized (lock) {
      System.out.println("Thread 2: Notifying...");
      lock.notify();
    } });

    t1.start();
    Thread.sleep(1000);
    t2.start();
  }   }
```

notify() vs notifyAll()

Comparison

Feature	notify()	notifyAll()
Threads awakened	One thread	All waiting threads
Selection	Random thread	All threads
Performance	Faster	Slower
Risk	May cause deadlock	Safer
Usage	When one thread is enough	When multiple threads depend

notify() vs notifyAll()

Comparison

Feature	notify()	notifyAll()
Threads awakened	One thread	All waiting threads
Selection	Random thread	All threads
Performance	Faster	Slower
Risk	May cause deadlock	Safer
Usage	When one thread is enough	When multiple threads depend

Key Point:

notify() vs notifyAll()

Comparison

Feature	notify()	notifyAll()
Threads awakened	One thread	All waiting threads
Selection	Random thread	All threads
Performance	Faster	Slower
Risk	May cause deadlock	Safer
Usage	When one thread is enough	When multiple threads depend

Key Point:

- `notify()` → wakes up only one thread

notify() vs notifyAll()

Comparison

Feature	notify()	notifyAll()
Threads awakened	One thread	All waiting threads
Selection	Random thread	All threads
Performance	Faster	Slower
Risk	May cause deadlock	Safer
Usage	When one thread is enough	When multiple threads depend

Key Point:

- `notify()` → wakes up only one thread
- `notifyAll()` → wakes up all waiting threads

notify() vs notifyAll()

Comparison

Feature	notify()	notifyAll()
Threads awakened	One thread	All waiting threads
Selection	Random thread	All threads
Performance	Faster	Slower
Risk	May cause deadlock	Safer
Usage	When one thread is enough	When multiple threads depend

Key Point:

- `notify()` → wakes up only one thread
- `notifyAll()` → wakes up all waiting threads

Use notifyAll() when multiple threads are waiting

Thank You!

Stay Connected

Premanand S

Email:	premanand.s@vit.ac.in
Phone:	+91-7358679961
LinkedIn:	linkedin.com/in/premsanand
Instagram:	instagram.com/premsanand
WhatsApp Channel:	anandsDataX
Google Scholar:	Google Scholar Profile
GitHub:	github.com/anandprems